

Hiero Workflow App Architecture-First Response

Pre-interview task | Darshit | LFDT Mentorship 2026

Thesis. Build a shared automation core in `sdk-automations`, expose it via a **GitHub Action adapter first**, retain Probot / GitHub App as the later orchestration path. SDKs keep local workflow ownership, security controls, and policy; reusable decision logic moves into the shared core.

Centralise: decision logic, schemas, fixtures, parity tests, adapters, releases. **Keep local:** workflow YAML, triggers, permissions, checkout, harden-runner, policy. **Never rewrite:** SDK security controls or maintainer governance.

Q1. Well-Architected Principles Applied to Hiero

[4 pts
K=2, R=2]

[K = 2] **Clear responsibility.** SDK repos own triggers, permissions, checkout, harden-runner, secrets, and local policy. The central repo owns automation decisions, schemas, adapters, fixtures, and releases. No step hides a security control or merges two governance domains.

[K = 2] **Secure by design.** Least-privilege `GITHUB_TOKEN`; no untrusted PR code in privileged jobs; pin actions by full commit SHA; CODEOWNERS on config; fail closed on invalid config.

[R = 2] **Low coupling / high cohesion.** Automation rules live in `packages/core`; GitHub Actions and Probot are thin adapters over the same behaviour no duplication of logic.

[R = 2] **Observable and testable.** Every automation supports dry-run output, structured logs, idempotent writes, golden fixtures, and parity tests before write mode is enabled upstream.

Evolvable. New automations are registered in the core and versioned via config schema; no cross-SDK script copying required.

Q2. Proposed Final System Architecture

[4 pts
R=2, A=2]
[Link](#)

[R = 2] [A = 2] Four zones: **Python / C++ SDK repos** (own all workflow control and security) **GitHub Action adapter** (current path; validates inputs, calls core, supports dry-run / writes) **packages/core** (registry, review-sync, assignment, PR checks, comment builders, label decisions, fixtures, parity tests) **GitHub API**. Probot / GitHub App calls the same core later no second behaviour implementation.

Invariant: centralise reusable decision logic; do not centralise repository control until governance, security, and parity gates are satisfied.

Q3. Architectural Boundaries and Why They Exist

[4 pts
R=2, E=2]

Boundary	What lives there and why
Repo-local	YAML, permissions, checkout, harden-runner, concurrency, token. Remain local because these are security and maintainer-ownership controls. Moving them hides the security surface from the team accountable for it.
Shared-core	Label decisions, assignment, review-sync, PR checks, comment builders, retry helpers, schemas, fixtures. Central because duplication creates maintenance cost and behavioural drift.
Policy	Labels, team handles, docs links, limits, enabled automations. Protected local config so SDK-specific choices do not become code forks in the shared repo.
Hosted-app	Probot hosting deferred: introduces webhook secrets, installation permissions, availability SLAs, incident response, and operational ownership not yet warranted.

[E = 2] **Key evaluation:** Each boundary maps to a real risk if crossed hiding permissions, creating hidden code forks, or adding unowned operational surface.

Q4. Tooling Choices

[6 pts
R=2, A=2, E=2]

[R = 2]	
Area	Recommendation and justification
Caller runtime	GitHub Actions first. Matches existing SDK ops; maintainers keep YAML, permissions, checkout, concurrency, and harden-runner local. No new hosting in Phase 1.
Core language	Node.js / Octokit. Native to GitHub automation; bundles cleanly as an action artefact; no secondary runtime for the adapter layer.
Shared logic	packages/core. Adapter-agnostic modules. Actions and Probot adapters call the same functions behaviour tested once, delivered twice.
Config	YAML / JSON + JSON Schema. Versioned contract; invalid config fails closed before any write occurs.
Verification	Unit + mock + fixtures + canaries + parity. Each layer catches a different failure class: logic, API contracts, or real-world call paths.
Security ops	Harden-Runner, CODEOWNERS, SHA pins, Dependabot, rollback docs. Governance gate, not optional extras.

[A = 2] [E = 2] Every choice is justified by what it *prevents*. Node.js bundles natively; JSON Schema enforces a contract. Tooling that cannot be pinned, audited, or rolled back is not acceptable in a security-sensitive automation path.

Q5. How This Solution Builds Iteratively

[2 pts
A=2]

- [A = 2]
- Architecture + reversible work in parallel.** Core skeleton, schema, fixtures, and dry-run path proceed now. Production writes wait for architecture gates.
 - Pilot Python review-sync first.** Bounded, canary-proven. One controlled write mode enabled after parity tests pass.
 - Map assignment logic before extracting it.** Same filenames do not guarantee identical policy; extraction without a behaviour map creates hidden drift.
 - C++ follows investigation.** Understand awkward workflows before introducing an unproven wrapper.
 - Probot comes after core is trusted** as an adapter over the validated contract, not a second implementation.

Q6. Malicious Pull Request Risks and Safeguards

[10 pts
K=4, R=3, A=3]

[K = 4] Highest-risk design area. The architecture assumes a malicious contributor controls: PR title / body, branch name, changed files, artefacts from untrusted code, and workflow / config changes in the PR branch.

Risk	Safeguard
pull_request_target abuse	Never checkout PR head in privileged jobs; load config from default branch; use job-level least privilege; avoid secrets in PR contexts.
Token / secret exfiltration	Harden-runner early in every job; restrict / audit egress; mask sensitive values; review logs during all canary runs.
Action / dependency compromise	Pin all actions by full commit SHA; bundle central actions; use Dependabot and <code>npm audit</code> continuously.
Config / policy tampering	CODEOWNERS on <code>.github/hiero-automation.*</code> ; schema-validate every invocation; reject unknown fields; fail closed on any error.
Artefact poisoning	Validate artefact identity: run source, run ID, path, checksum. Never treat artefacts from untrusted PR code as an authority signal.
Bot loops / broad writes	Actor / bot guards, idempotent markers, scoped label allowlists, concurrency keys, rollback by disabling the caller workflow or pinning back a SHA.

[R = 3] [A = 3]

Systemic safeguards: validate actor type, event type, repo, PR number, label allowlists, and config schema *before* acting unknown config fails closed. All writes are idempotent and scoped to known bot markers only. For `workflow_run`: validate artefact identity, run source, and expected paths locally before the action consumes anything. For Probot (later): verify webhook signatures, use installation-scoped permissions, store secrets outside repos, implement rate-limit handling, and provide a clear disable / rollback path.

Q7. Configuration Standardisation vs. User Input [6 pts

K=2, R=2, A=2]

[K = 2]	[R = 2]
Standardise centrally	Allow per-repo guardrails (with)
Schema version, automation names, dry-run / write contract, result / log format, bot comment markers, retry logic, error categories, release compatibility, permission docs.	Label names, team handles, docs links, support channels, assignment limits, enabled automations, schedules, branch filters, repo-specific message links.
[A = 2] Rule of thumb: common behaviour → core; repo policy → protected config validated by the central schema; one-off workflow / security constraint → keep strictly local. Avoid arbitrary scripts, unlimited switches, and template overrides those recreate the maintenance tax centrally.	

Q8. Community Leverage Plan [10 pts

K=3, R=3, A=4]

[K = 3]

Public issue backlog. Architecture becomes a contribution surface covering behaviour mapping, fixture collection, docs, config examples, golden tests, canary workflows, security review, and SDK adoption guides useful work that does not require maintainers to triage risky rewrites.

[R = 3]

Skill-based labels: `good-first-issue` (docs / fixtures) · `beginner` (tests / samples) · `intermediate` (adapters / config) · `advanced` (security pilots / C++ investigation).

[A = 4]

RFC per automation migration. Each RFC states: reusable logic, repo-local boundary, policy config, fixtures, rollback plan, and owner. No production behaviour change without dry-run proof, parity tests, a security checklist, and maintainer approval.

References: arc42.org · GitHub Actions secure-use docs · StepSecurity Harden-Runner · LFD T Issue #73 · C++ Issue #1627 · Richards, *Software Architecture Patterns* (O'Reilly, 2015)

Small, reviewable PRs only.

Work package	Good community issue shape
Behaviour mapping	Compare Python and C++ for one automation; record common logic, policy differences, fixtures, and unknowns.
Fixture / parity tests	Convert real examples to golden cases so central behaviour is proved before writes.
Docs / examples	Caller workflow examples, config samples, rollback instructions, security checklists.
Canary support	Run dry-run pilots, collect logs, identify false positives, file follow-up issues.
Security review	Threat-model <code>pull_request_target</code> , artefact flows, permissions, installation scopes, and release pinning.

Demos and office hours. Publish canary reproduction steps so contributors can verify call paths independently. Python / C++ maintainers review for behaviour; central maintainers review for schema, release, and security.

Q9. Architecture vs. Immediate Development [4 pts

R=2, E=2]

[R = 2]		
Option	Risk	Assessment
Skip architecture	High	Replicates existing problem: independent evolution, inconsistent security, no parity baseline.
Full freeze	Medium	Delays reversible work unnecessarily; fixtures, schema, and dry-run adapters carry no risk.
2-week arch + reversible work	Low	Recommended. Architecture gates production writes; reversible work runs in parallel.
[E = 2] Allocation: Wk 1 context diagram, boundary definitions, threat model, config schema contract. Wk 2 behaviour maps, pilot criteria, rollback plan, community issue backlog. Safe to begin now: fixtures, schema validation, core modules, dry-run adapters, docs. Must wait: upstream write mode, SDK security control changes, C++ wrapper introduction.		